Project - High Level Design

on

Construction Planning Assistant Agent

Course Name: Agentic AI

INSTITUTION NAME: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01.	PALAK HARDE	EN22CS301673
02.	PARIKSHIT GOLE	EN22CS301685
03.	NITIN KUMAR SINGH	EN22CS301662
04.	NITISH SINGH YADAV	EN22CS301663
05.	PRIYAL GUPTA	EN22CS301619

Group Name: Group 07D6

Project Number: AAI-18

Industry Mentor Name:

University Mentor Name:

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

Scope of the Document

This document presents the High Level Design (HLD) of the Construction Planning Assistant Agent. It describes the overall system architecture, major components, data flow, and design considerations involved in building the agent.

Intended Audience

This document is intended for project evaluators, developers, academic mentors, and stakeholders who need an overview of the system design and working principles.

System Overview

The Construction Planning Assistant Agent is an intelligent planning system designed to assist engineers in organizing construction tasks. The system accepts a high-level goal and automatically decomposes it into structured tasks, validates resource availability, and generates an

optimized execution schedule.

The agent follows a modular architecture consisting of a Planner Agent, Task Decomposer, Dependency Manager, Resource Validator, and Scheduler module.

2. System Design

3.

2.1 Application Design

The Construction Planning Working Agent is designed as a modular, tool-integrated AI system that automates construction task planning.

The application follows an Agent + Tools Architecture, where:

The LLM (via OpenRouter API) performs reasoning and task decomposition.

Internal tools (Resource Validator, Scheduler) execute deterministic validation and scheduling logic.

The system ensures dependency correctness, resource feasibility, and optimized execution sequencing.

2.2 Process Flow

Construction Planning Working Agent

The Process Flow explains how the system transforms a high-level construction goal into a validated and optimized execution schedule. It describes the step-by-step movement of control from input to final output.

• Step 1: Goal Submission

The process begins when the user provides a high-level construction objective, such as building a foundation or constructing a residential unit. This goal is submitted to the system through the user interface. The input at this stage is unstructured natural language.

• Step 2: Intelligent Task Decomposition

The Planning Agent receives the goal and sends a structured prompt to the Large Language Model using the OpenRouter API. The LLM analyzes the construction objective and decomposes it into smaller actionable tasks. It also identifies estimated durations, required resources, and logical dependencies

between tasks. The output is generated in a structured format for further processing.

• Step 3: Task Structuring and Formatting

The system converts the LLM response into internal task objects. Each task is assigned attributes such as task ID, duration, required resources, and dependency list. This ensures the data is consistent and machine-readable before validation begins.

• Step 4: Dependency Validation

The Dependency Checking module verifies that tasks follow a logical execution sequence. It ensures that prerequisite tasks are completed before dependent tasks begin. Circular dependencies or invalid sequencing are detected and corrected at this stage to maintain structural integrity.

• Step 5: Resource Validation

After dependencies are confirmed, the tasks are passed to the Resource Validator tool. This module checks whether sufficient labor, materials, and equipment are available for each task. If resource shortages are detected, the system prevents unrealistic scheduling and flags the issue for correction.

• Step 6: Task Scheduling

Once tasks are validated, they are forwarded to the Scheduler component. The scheduler assigns start and end times while respecting dependencies and resource constraints. It ensures that resources are not over-allocated and attempts to minimize overall project duration.

• Step 7: Plan Generation and Output

After scheduling is complete, the system generates the final construction plan. The output includes an ordered task list, timeline details, and resource allocation information. This optimized and validated plan is then presented to the user.

2.3 Information Flow

Construction Planning Working Agent

The Information Flow describes how data moves across different components of the system and how it is transformed at each stage. It ensures that raw user input is gradually converted into a structured, validated, and executable construction plan.

- User Input to Planning Agent

The information flow begins when the user submits a high-level construction goal in natural language form. This input is received by the User Interface and forwarded to the Planning Agent. At this stage, the information is unstructured and purely textual.

- Planning Agent to LLM (via OpenRouter API)

The Planning Agent transforms the user input into a structured prompt and sends it to the Large Language Model using the OpenRouter API. Along with

the construction goal, formatting instructions are included to ensure structured output.

The LLM processes this request and returns decomposed tasks, estimated durations, required resources, and dependency relationships in a structured format.

- LLM Output to Task Structuring Module

The structured response received from the LLM is converted into internal task objects. Each task is assigned attributes such as task ID, duration, dependency list, and required resources. This transformation ensures that the AI-generated data becomes system-readable and ready for validation.

- Task Data to Dependency Checker

The task objects are passed to the Dependency Checking module. This module reads task IDs and dependency mappings to validate logical sequencing. If invalid dependencies or circular references are found, corrections are applied before proceeding further.

- Validated Tasks to Resource Validator

After dependency validation, task information flows to the Resource Validator tool. The module compares task resource requirements with available resource data stored in the system. It determines whether the plan is feasible from a resource perspective and updates task feasibility status accordingly.

- Feasible Tasks to Scheduler

Once resources are validated, the Scheduler receives task details including duration, dependencies, and resource availability. It calculates start and end times while ensuring that dependencies are respected and resources are not over-allocated. The information is transformed into a time-based execution plan.

- Final Plan to User Interface

The scheduled and validated task data is then passed back to the User Interface. The system presents the final construction plan, including task order, timeline, and resource allocation details, in a structured and readable format.

2.4 Components Design

Construction Planning Working Agent

The Component Design of the Construction Planning Working Agent focuses on how individual modules are structured and how they interact with each other to transform a high-level construction goal into a validated and scheduled plan. The system is designed in a modular way so that each component has a clear responsibility and can operate independently while still collaborating with other components.

• User Interface Component

The User Interface component acts as the entry point of the system. It accepts the high-level construction goal from the user in natural language form. This component is responsible only for collecting input and displaying the final output. It does not perform reasoning or scheduling logic. Its main role is to pass the user's request to the Planning Agent and later present the generated execution plan in a readable format.

• Planning Agent Component (Core Logic Layer)

The Planning Agent is the central brain of the system. It receives the construction goal and prepares a structured prompt to send to the Large Language Model through the OpenRouter API. This component is responsible for converting raw user input into a format suitable for AI reasoning. It also ensures that the output from the LLM is structured properly before passing it to other modules. The Planning Agent does not directly schedule tasks but coordinates the entire workflow.

• Task Decomposition Module

This module processes the structured response received from the LLM. It extracts task details such as task name, duration, required resources, and dependencies. The goal of this component is to transform AI-generated text into well-defined internal task objects that the system can process further. It ensures that the data format is consistent and ready for validation.

• Dependency Checking Module

The Dependency Checking component ensures logical correctness of the generated tasks. It verifies that tasks are arranged in a valid execution order and that no circular dependencies exist. This module guarantees that a task will not start before its prerequisite tasks are completed. By doing so, it maintains structural integrity in the construction workflow.

• Resource Validator Tool

The Resource Validator is a deterministic rule-based component. It checks whether sufficient labor, materials, and equipment are available for each task. This module compares task requirements with predefined available resources stored in the system. If resources are insufficient, it flags the issue and prevents unrealistic scheduling. This ensures that AI-generated plans are grounded in practical constraints.

• Scheduler Component

The Scheduler is responsible for assigning start and end times to tasks. It considers task dependencies and validated resource availability to generate an optimized timeline. The scheduler ensures that tasks do not overlap in a way that causes resource conflicts. It also attempts to minimize total project duration while respecting all constraints. This component transforms validated tasks into a complete execution schedule.

• OpenRouter Integration Component

This component manages communication with the OpenRouter API. It handles API key management, request formatting, response handling, and error management. It ensures secure interaction with the Large Language Model and provides structured AI outputs to the Planning Agent. This separation keeps AI integration isolated from business logic, improving maintainability.

2.5 Key Design Considerations

Construction Planning Working Agent

The design of the Construction Planning Working Agent was guided by several important technical and architectural considerations to ensure reliability, scalability, and practical usability in real-world construction planning scenarios.

• Separation of AI Reasoning and Deterministic Logic

One of the most important design considerations was separating AI-based reasoning from rule-based validation. The Large Language Model (via OpenRouter) is used only for intelligent task decomposition and planning logic. However, dependency validation, resource checking, and scheduling are handled by deterministic tools. This separation prevents over-reliance on AI and reduces the risk of hallucinated or impractical outputs. It ensures that all final plans are logically and operationally feasible.

• Modular Architecture for Scalability

The system is designed in a modular manner, where each component (Planning Agent, Dependency Checker, Resource Validator, Scheduler) operates independently. This makes the system easy to scale and extend. For example, additional modules such as Cost Estimation, Risk Analysis, or Safety Compliance can be added in the future without redesigning the entire system. Modularity also improves maintainability and debugging efficiency.

• Structured Data Flow and Validation

Since the LLM generates outputs dynamically, strict structured formatting was enforced. The Planning Agent ensures that AI responses are converted into structured task objects before being processed further. Validation layers are placed after each major stage (dependency checking, resource validation) to prevent error propagation. This layered validation ensures system robustness.

• Dependency Integrity and Conflict Prevention

Construction planning requires strict sequencing of activities. A key design focus was preventing circular dependencies and invalid task orders. The dependency checking module ensures that every task has a logically valid prerequisite structure before moving to scheduling. This prevents unrealistic execution flows.

• Resource Constraint Awareness

Real-world construction projects are limited by labor, materials, and equipment. Therefore, the system was designed to validate resource availability before scheduling. The Resource Validator ensures that no plan is generated without sufficient resources. This makes the output practical and implementable.

• Optimization of Project Timeline

The Scheduler is designed to minimize overall project duration while respecting dependencies and resource constraints. Efficient sequencing and controlled resource allocation help in generating an optimized execution plan. This consideration ensures that the system produces not only valid but also efficient schedules.

• Error Handling and Fault Tolerance

The system includes structured error handling at multiple stages. API failures, invalid task structures, dependency conflicts, or resource shortages are detected early. Instead of failing silently, the system provides corrective feedback. This improves reliability and user trust.

• Security of API Integration

Since the system uses the OpenRouter API, secure handling of API keys was considered essential. API keys are stored in environment variables rather than hardcoded in the application. This prevents credential leakage and enhances security.

• Extensibility for Future Enhancements

The architecture is designed to support future enhancements such as multi-project planning, real-time resource tracking, integration with IoT-based construction monitoring systems, or advanced optimization algorithms. The current design lays a scalable foundation for such future upgrades.

2.6 API Catalogue

Construction Planning Working Agent

The API Catalogue defines the key APIs used in the Construction Planning Working Agent system. These APIs enable communication between components and allow interaction with external services such as the OpenRouter LLM.

1. Create Construction Plan API

Endpoint: /api/plan

Method: POST

This API is used to generate a construction plan from a high-level goal. The user sends a construction objective in the request body. The system processes the request, decomposes tasks, validates dependencies and resources, schedules them, and returns the final plan.

Request Example:

```
{
  "goal": "Construct a residential building foundation"
}
```

Response Example:


```
{
  "project_duration": "15 days",
  "tasks": [
    {
      "task_id": "T1",
      "name": "Site Clearing",
      "start_day": 1,
      "end_day": 2
    }
  ]
}
```


2. Resource Availability API

Endpoint: /api/resources

Method: GET

This API returns available labor, equipment, and material details stored in the system. It is mainly used internally by the Resource Validator tool.

Response Example:


```
{
  "labor": 10,
  "excavators": 2,
  "cement_bags": 200
}
```

3. Validate Resources API

Endpoint: /api/validate-resources

Method: POST

This API checks whether required resources for a given task list are available. It returns validation status and highlights shortages if any exist.

4. Scheduler API

Endpoint: /api/schedule

Method: POST

This API takes validated tasks with dependencies and generates a time-based schedule. It assigns start and end times while respecting constraints.

5. OpenRouter Integration API (External API)

Service Used: OpenRouter API

This external API is used to access the Large Language Model for intelligent task decomposition. The Planning Agent sends the construction goal and receives structured task breakdown data.

Authentication is handled using an API key stored securely in environment variables.

4. Data Design

Data Design—> Construction Planning Working Agent

The Data Design defines how data is structured, stored, accessed, maintained, and managed throughout the lifecycle of the Construction Planning Working Agent.

3.1 Data Model

The system uses a structured data model to represent construction projects and tasks in a consistent format. The core entity in the system is the Project, which contains multiple Tasks.

Each task includes attributes such as task ID, task name, duration, required resources, dependencies, start time, and end time. Dependencies are stored as references to other task IDs, allowing logical sequencing. Resource data includes available labor, equipment, and materials.

The data model ensures that AI-generated outputs are converted into structured objects before processing. This structured approach makes validation, scheduling, and optimization easier and prevents inconsistencies. The design supports future extension, such as adding cost estimation or risk assessment fields without major redesign.

3.2 Data Access Mechanism

The system follows a controlled data access mechanism to maintain consistency and integrity.

Task data and resource data are accessed through defined service layers or APIs rather than directly modifying storage. The Planning Agent retrieves and updates task objects only through structured functions. The Resource Validator reads available resource data before performing validation. The Scheduler reads validated task data and writes back scheduling results.

This layered data access approach prevents unauthorized modification, reduces coupling between components, and ensures clean separation between business logic and data handling.

3.3 Data Retention Policies

The system follows simple and practical data retention policies.

Project plans can be stored for future reference, audit, or modification. Temporary intermediate data such as raw LLM responses may not be permanently stored unless required for debugging or monitoring.

Sensitive data such as API keys (OpenRouter key) are never stored in application code or databases. They are securely maintained in environment variables.

Data retention duration can be configured depending on organizational needs. For example, completed project plans may be archived after a defined period to reduce storage load.

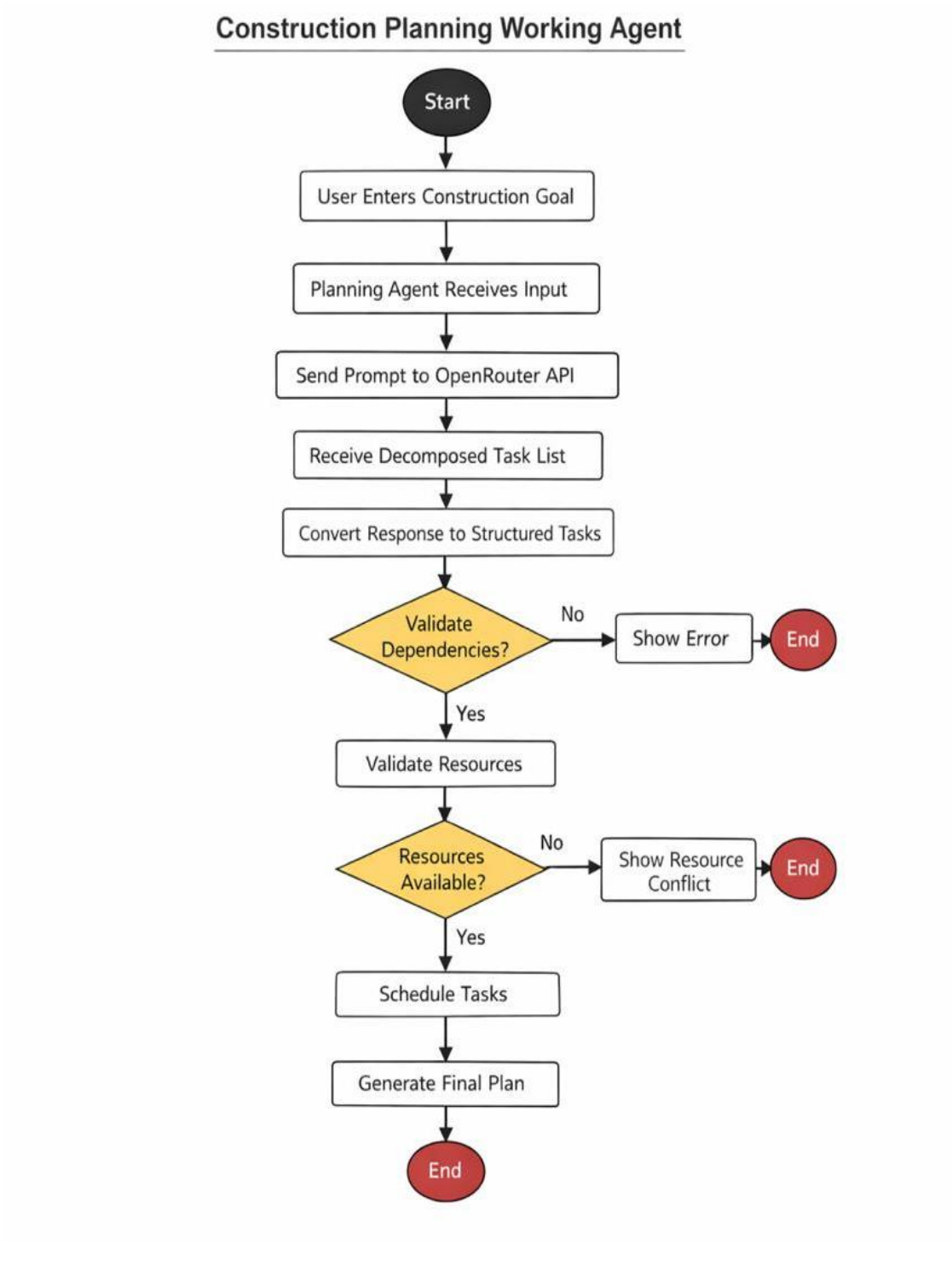
3.4 Data Migration

The system design allows future data migration if storage mechanisms change.

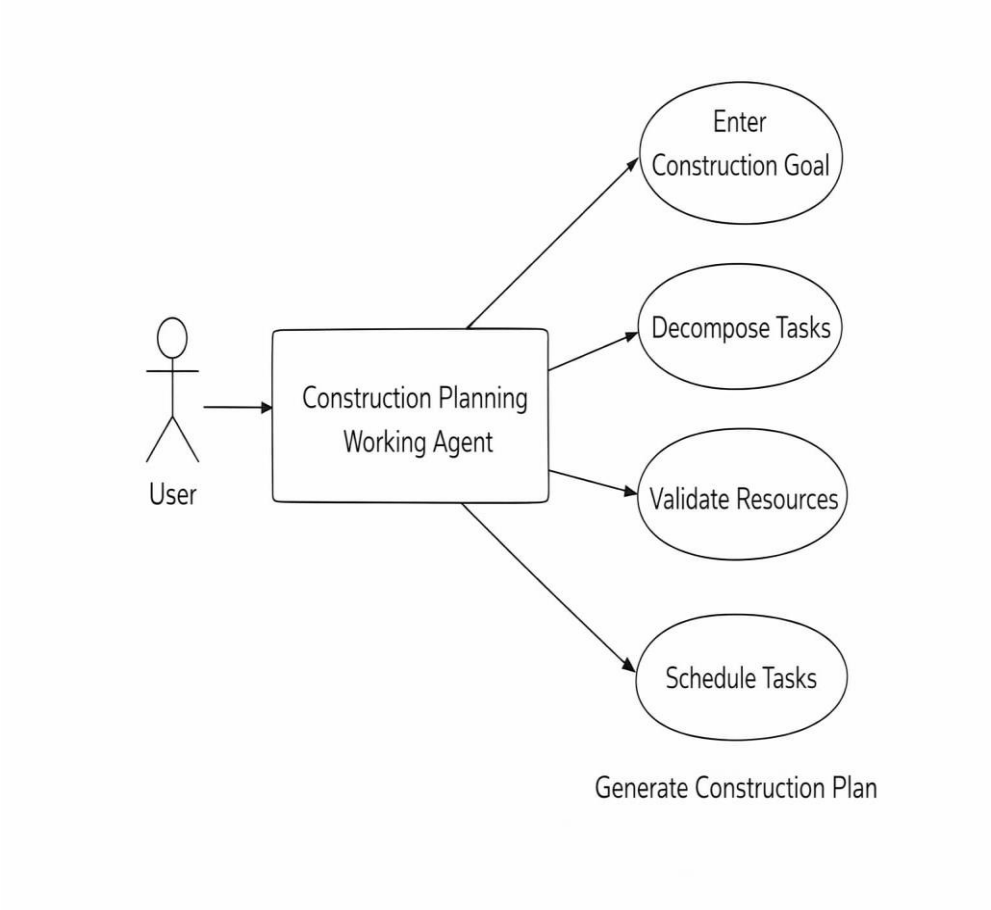
If the application transitions from in-memory storage to a database system, structured task and project models can be mapped directly to database schemas. Since the data model is already organized in structured formats (such as JSON objects), migration to relational or NoSQL databases becomes straightforward.

Versioning strategies can also be implemented if the data structure evolves in the future. This ensures backward compatibility and smooth system upgrades without data loss.

- **ACTIVITY DIAGRAM**



- **CLASS DIAGRAM**



4. Interfaces

Construction Planning Working Agent

The Interfaces section describes how different parts of the system interact with users, internal components, and external services. Proper interface design ensures smooth communication and clear separation of responsibilities.

4.1 User Interfaces

The User Interface serves as the interaction layer between the user and the Construction Planning Working Agent. It allows users to submit high-level construction goals in natural language form and receive structured execution plans as output.

The interface may be a command-line interface (CLI) or a simple web-based interface. Users enter a construction objective such as building a foundation or planning structural work. The system then processes the input and displays the final plan, including ordered tasks, timelines, and resource allocation details.

The user interface is designed to remain simple and focused. It does not contain planning logic but only handles input collection and output presentation.

4.2 System Interfaces

System Interfaces define how internal components communicate with each other.

The Planning Agent interacts with the Task Decomposition module, Dependency Checker, Resource Validator, and Scheduler through well-defined function calls or APIs. Each component passes structured task objects rather than raw text.

For example, after receiving decomposed tasks from the LLM, the Planning Agent forwards structured task data to the Dependency Checking module. Once validated, tasks are passed to the Resource Validator and then to the Scheduler.

This structured internal communication ensures modularity, reduces coupling, and makes debugging and future enhancements easier.

4.3 External Interfaces

External Interfaces define how the system interacts with outside services.

The primary external interface in this system is the integration with the OpenRouter API. The Planning Agent sends structured prompts to the OpenRouter service and receives task decomposition responses from the Large Language Model.

Authentication with the external service is handled using a securely stored API key in environment variables. The system manages request formatting, response parsing, and error handling while interacting with the external API.

This external interface enables intelligent reasoning capabilities while keeping AI integration separate from core validation and scheduling logic.

5. State and Session Management

Although the system does not use an external database, it maintains an internal in-memory data store that holds predefined resource information (such as available labor, equipment, and materials). Therefore, the system is not fully persistent, but it does manage temporary internal state during execution.

5.1 Stateless Processing with Internal In-Memory State

The system follows a largely stateless request-processing model, meaning each construction goal is processed independently from start to finish. However, it relies on a small internal "database" of hard-coded available resources stored in memory.

When a request is received, the system:

Reads the available resource data (e.g., number of workers, equipment units, materials).

Uses this data during validation and scheduling.

May temporarily update resource availability during scheduling simulation.

These changes exist only during execution. Once the process completes, the internal resource configuration resets to its original hard-coded values.

Since there is no external database, no project data or resource updates are permanently stored. Every new request starts with the same base resource configuration.

5.2 Session Handling During Execution

During the lifecycle of a single planning request, the system creates a temporary in-memory session. This session holds:

Decomposed task list from the LLM

Dependency relationships

Resource validation results

Temporary resource allocation updates

Scheduled start and end times

The Scheduler may reduce available resources while assigning tasks to simulate real-world allocation. However, this modification is temporary and exists only within the execution context.

Once the final construction plan is generated and returned to the user, all intermediate session data is cleared. No state is retained for future requests.

5.3 Internal Resource Data Handling

The hard-coded resource data acts as a lightweight internal data source. It is structured in predefined variables or configuration files within the application.

This design keeps the system simple while still enabling realistic validation. However, since the data is not persisted externally:

Resource updates are not saved across sessions.

Multi-user concurrency is limited.

Historical project tracking is not supported.

Future enhancements could include integrating a proper database to enable persistent resource tracking and session continuity.

6. Cache

1. In-Memory Caching

The system can implement basic in-memory caching during runtime. When a user submits a construction goal, the generated plan can be temporarily stored in memory along with the input goal.

If the same goal is requested again during the application runtime, the system can return the cached result instead of:

Calling the OpenRouter API again

Re-running dependency validation

Re-executing resource validation

Re-scheduling tasks

This reduces API usage cost and improves response speed.

Since there is no database, this cache exists only while the application is running. Once the application restarts, the cache is cleared.

2. LLM Response Caching

OpenRouter API calls are computationally expensive and may introduce latency. To optimize performance, the system can cache structured LLM responses for repeated prompts.

If two users submit identical construction goals, the cached LLM output can be reused. This reduces API calls and ensures faster task decomposition.

However, cache invalidation must be handled carefully. If resource configurations change internally, the cached plan may no longer be valid.

3. Resource Data Caching

Since available resources are hard-coded and stored internally, they are already effectively cached in memory. Accessing them does not require external queries. This improves validation speed and keeps the system lightweight.

4. Cache Limitations

Because the system does not use persistent storage:

Cached plans are lost when the application restarts.

No long-term storage of project plans exists.

Cache does not support distributed or multi-instance environments.

Future improvements could include integrating a distributed caching mechanism (such as Redis) if scalability becomes a requirement.

7. Non-Functional Requirements

Construction Planning Working Agent

1. Security Aspects

Security is an important consideration in the Construction Planning Working Agent, especially because it integrates with an external AI service (OpenRouter API).

The OpenRouter API key is stored securely using environment variables rather than being hard-coded inside the application. This prevents accidental exposure

of credentials in source code repositories. Access to the API key is restricted to the runtime environment only.

All communication with the OpenRouter service is performed through authenticated API requests. The system ensures that sensitive configuration details are not logged or exposed in error messages.

Since the application does not use a database and does not store user-sensitive data, the risk of data breaches is minimized. Intermediate task data and session information exist only in memory and are cleared after processing completes.

Input validation is also considered a security measure. User-provided construction goals are treated as plain text and processed safely, reducing the risk of injection attacks or malformed input affecting system behavior.

Overall, the system follows secure configuration management, minimal data storage, and safe API handling practices to ensure operational security.

2. Performance Aspects

Performance in the Construction Planning Working Agent depends primarily on two factors: AI processing latency and internal validation efficiency.

The task decomposition stage depends on the OpenRouter API response time. While this introduces external latency, the remaining modules (dependency checking, resource validation, scheduling) operate entirely in memory and execute quickly.

The system follows a stateless design, which improves performance by avoiding database reads and writes. Since no persistent storage is involved, processing remains lightweight and fast.

Internal resource data is hard-coded and stored in memory, enabling instant access during validation. The scheduler operates using efficient sequencing logic, ensuring that even multiple tasks can be processed without significant delay.

If caching is implemented for repeated goals, performance can be further optimized by reducing repeated LLM calls.

Overall, the system is optimized for small to medium-sized construction projects, ensuring fast execution, low overhead, and efficient processing within a single runtime session.

8. References

- OpenRouter API Documentation – https://openrouter.ai/docs
- Python Official Documentation – https://docs.python.org/3/
- OpenAI API Reference (Chat Completions) – https://platform.openai.com/docs/api-reference/chat
- LangChain Documentation – https://python.langchain.com/docs/